

JAVASCRIPT PROGRAMMING LAB (BCS156)

B.Tech CSE 1st SEMESTER

SUBMITTED TO:

MR. AYUSH KUMAR

SUBMITTED BY:

DEEPAK YADAV

STUDENT ID: BCS2026491

SN.	PROGRAM NAME	PAGE	DATE	SIGNATURE
1	Your First JavaScript Script	3		
2	Variables and Data Types	6		
3	Arithmetic, Assignment and Increment Operators	8		
4	Comparison Operators and Text	10		
5	Numbers, Math and Type Conversion	12		
6	Conditions and the Ternary Operator	14		
7	Switch and If-Else Grades	17		
8	For, While and Do-While Loops	20		
9	Nested Loops and Continue	23		
10	Functions and Return Values	26		
11	Finding and Fixing Errors	29		
12	Calculator Functions	32		
13	The BMI Calculator Page Layout	35		
14	Script Linking and BMI Functions	39		
15	Testing Normal, Boundary and Invalid Inputs	42		

EXPERIMENT NO. 1

1. Experiment Title:

Your First JavaScript Script

2. Aim:

To print messages with console.log and to link a script file to a web page.

3. Software / Tools Required:

JavaScript (ES6), HTML5, Visual Studio Code (editor), Google Chrome browser (DevTools Console)

4. Theory:

console.log() writes a message to the console and is the simplest way to see the output of a JavaScript program. A JavaScript file can be linked to a web page with the <script src="..."> tag; placing the tag just before the closing </body> tag makes sure the page content is loaded before the script runs. A comment (// ...) is ignored by JavaScript and is used to explain the code.

5. Algorithm / Procedure:

Task 1: Print the name and the course name with console.log.

Step 1: Open a new file and save it as assignment1_task1.js.
Step 2: Write console.log("Neha") to print the name.
Step 3: Write console.log("JavaScript Programming lab") on the next line to print the course name.
Step 4: Run the file and check the output.

Task 2: Create a page titled My First Page and link intro.js to it.

Step 1: Create assignment1_task2.html with the title and the heading My First Page.
Step 2: Create intro.js and write two console.log lines in it.
Step 3: Add <script src="intro.js"></script> just before the closing </body> tag.
Step 4: Open the HTML file in a browser, press F12 and open the Console tab.

Task 3: Print the name and the course name with a comment above each console.log line.

Step 1: Save a new file as assignment1_task3.js.
Step 2: Write a comment saying what the next line prints, then write the first console.log.
Step 3: Write a comment for the second line, then the second console.log.
Step 4: Run the file and check the output.

6. Program / Queries / Commands:

Task 1

File: assignment1_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 1  Task: 1

console.log("Neha");
console.log("JavaScript Programming lab");
```

Task 2

File: assignment1_task2.html

```
<!-- Name: Deepak Yadav | Student ID: BCS2026491 | Assignment: 1  Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>My First Page</h1>

  <script src="intro.js"></script>
</body>
</html>
```

File: intro.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 1  Task: 2

console.log("Neha");
console.log("JavaScript Programming lab");
```

Task 3

File: assignment1_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 1  Task: 3

// This line prints the name of the student
console.log("Neha");
// This line prints the name of the course
console.log("JavaScript Programming lab");
```

7. Input:

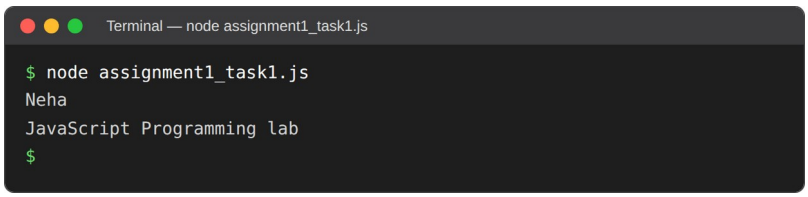
Task 1: No input is required. The two text values are written directly in the program.

Task 2: No input is required.

Task 3: No input is required.

8. Output:

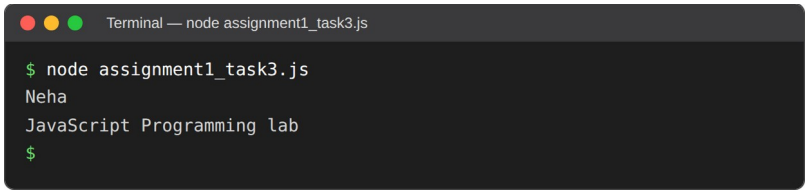
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. The name and course name were printed with console.log, and an external script file (intro.js) was linked to a web page and its output was seen in the browser console.

EXPERIMENT NO. 2

1. Experiment Title:

Variables and Data Types

2. Aim:

To store values in let, const and var variables, and to check the type of a value with typeof.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

A variable stores a value in memory. let declares a variable whose value can change, const declares a constant whose value cannot be changed, and var is the older way of declaring a variable. The basic data types are string (text), number, boolean (true/false) and undefined (a variable that has no value yet). The typeof operator returns the type of a value as a string.

5. Algorithm / Procedure:

Task 1: Declare the student name with let and the roll number with const, then print both.

Step 1: Declare let studentName = "Neha".
Step 2: Declare const rollNumber = 2401.
Step 3: Print both variables with console.log.

Task 2: Print the type of four values with typeof.

Step 1: Declare a variable with let but do not give it a value.
Step 2: Use typeof on the text "hello", the number 42, the value true and the empty variable.
Step 3: Print every result with console.log.

Task 3: Declare a variable subject with var and print it.

Step 1: Declare var subject and give it the value "Maths".
Step 2: Print the variable with console.log.

6. Program / Queries / Commands:

Task 1

File: assignment2_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 2  Task: 1

let studentName = "Neha";
const rollNumber = 2401;

console.log(studentName);
console.log(rollNumber);
```

Task 2

File: assignment2_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 2  Task: 2

let notAssigned;   // declared but has no value

console.log(typeof "hello");
console.log(typeof 42);
console.log(typeof true);
console.log(typeof notAssigned);
```

Task 3

File: assignment2_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 2  Task: 3

var subject = "Maths";
console.log(subject);
```

7. Input:

Task 1: studentName = Neha, rollNumber = 2401

Task 2: Values checked: "hello", 42, true and a let variable without a value.

Task 3: subject = Maths

8. Output:

Task 1

```
Terminal — node assignment2_task1.js

$ node assignment2_task1.js
Neha
2401
$
```

Task 2

```
Terminal — node assignment2_task2.js

$ node assignment2_task2.js
string
number
boolean
undefined
$
```

Task 3

```
Terminal — node assignment2_task3.js

$ node assignment2_task3.js
Maths
$
```

9. Result:

The experiment was performed successfully. Variables were declared with let, const and var, and the data type of different values was checked using typeof.

EXPERIMENT NO. 3

1. Experiment Title:

Arithmetic, Assignment and Increment Operators

2. Aim:

To use arithmetic, assignment and increment operators and to read the results.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

Arithmetic operators (+, -, *, /, %) perform calculations; the % operator gives the remainder of a division. JavaScript follows operator precedence, so * is done before +. Assignment operators such as += and -= change a variable and store the new value in it. The increment operator ++ increases a variable by 1.

5. Algorithm / Procedure:

Task 1: Print the results of 10 + 5 * 2 and 17 % 5 in the form expression = result.

Step 1: Evaluate 10 + 5 * 2 (multiplication is done first).
Step 2: Evaluate 17 % 5 (remainder of the division).
Step 3: Print each line as expression = result.

Task 2: Use += and -= on a variable total.

Step 1: Declare let total = 100.
Step 2: Add 20 with total += 20 and print total.
Step 3: Subtract 10 with total -= 10 and print total again.

Task 3: Use the increment operator on a counter.

Step 1: Declare let count = 5.
Step 2: Use count++ once.
Step 3: Print count.

6. Program / Queries / Commands:

Task 1

File: assignment3_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 3  Task: 1

console.log("10 + 5 * 2 =", 10 + 5 * 2);
console.log("17 % 5 =", 17 % 5);
```

Task 2

File: assignment3_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 3  Task: 2

let total = 100;

total += 20;
console.log(total);

total -= 10;
console.log(total);
```

Task 3

File: assignment3_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 3  Task: 3

let count = 5;
count++;
console.log(count);
```

7. Input:

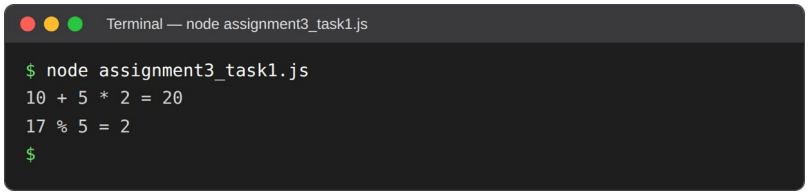
Task 1: Expressions: 10 + 5 * 2 and 17 % 5

Task 2: total = 100

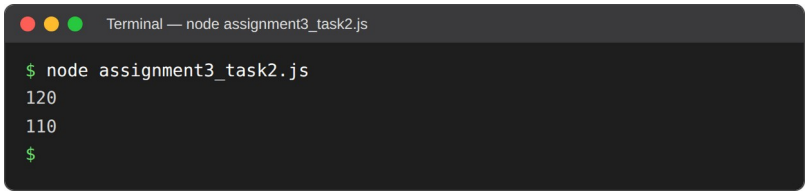
Task 3: count = 5

8. Output:

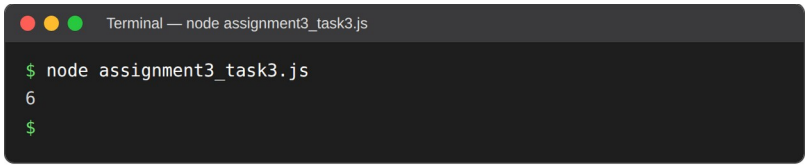
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. Arithmetic, assignment (+=, -=) and increment (++) operators were used and their results were printed.

EXPERIMENT NO. 4

1. Experiment Title:

Comparison Operators and Text

2. Aim:

To compare two values and to print text with template literals and string methods.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

The loose equality operator == compares only the values and converts the types, so 5 == "5" is true. The strict equality operator === compares both the value and the type, so 5 === "5" is false. A template literal is written between backticks and can put variables inside the text with ${ }. String methods such as trim() (removes spaces from both ends) and includes() (checks if some text is present) work on text.

5. Algorithm / Procedure:

Task 1: Compare 5 and '5' with == and ===.

Step 1: Evaluate 5 == '5' (compares only the value).
Step 2: Evaluate 5 === '5' (compares value and type).
Step 3: Print each line in the form expression: result.

Task 2: Print a welcome message with a template literal.

Step 1: Store the name in a variable and the subject in another variable.
Step 2: Write a template literal using backticks and ${ } for both variables.
Step 3: Print the message.

Task 3: Use trim() and includes() on a text.

Step 1: Store the text ' JavaScript is fun to learn ' in a variable.
Step 2: Print text.trim() to remove the extra spaces.
Step 3: Print text.includes('fun') to check whether the word is present.

6. Program / Queries / Commands:

Task 1

File: assignment4_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 4  Task: 1

console.log("5 == '5':", 5 == '5');
console.log("5 === '5':", 5 === '5');
```

Task 2

File: assignment4_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 4  Task: 2

let name = "Neha";
let subject = "Maths";

console.log(`Welcome, ${name}. Your subject is ${subject}.`);
```

Task 3

File: assignment4_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 4  Task: 3

let text = ' JavaScript is fun to learn ';

console.log(text.trim());
console.log(text.includes('fun'));
```

7. Input:

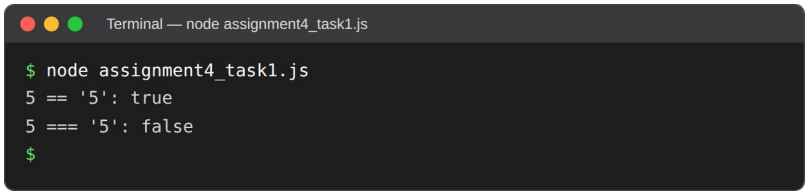
Task 1: Values compared: the number 5 and the text '5'

Task 2: name = Neha, subject = Maths

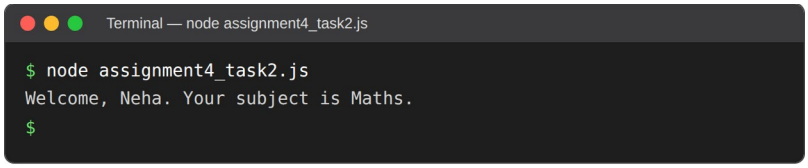
Task 3: text = ' JavaScript is fun to learn '

8. Output:

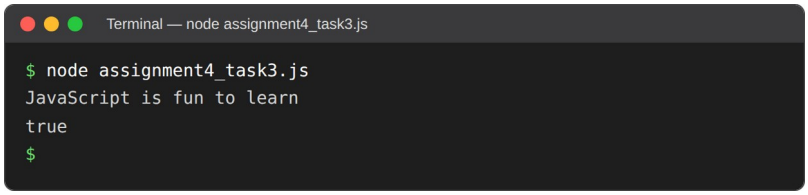
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. == and === were compared, a template literal was used to build a message, and the string methods trim() and includes() were applied.

EXPERIMENT NO. 5

1. Experiment Title:

Numbers, Math and Type Conversion

2. Aim:

To convert text to numbers and to use the Math object on numbers.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

Text can be converted to numbers with parseInt() (whole numbers) and parseFloat() (decimal numbers); both read the number from the start of the text and ignore the rest, so parseInt('42px') gives 42. The Math object has ready-made methods such as Math.round() (nearest whole number) and Math.max() (largest of the given numbers). Text must be converted before calculations, otherwise the result may be wrong.

5. Algorithm / Procedure:

Task 1: Convert text to numbers with parseInt and parseFloat.

Step 1: Use parseInt('42px') to read the whole number.
Step 2: Use parseFloat('3.5') to read the decimal number.
Step 3: Print both results with the labels parseInt: and parseFloat:.

Task 2: Use Math.round and Math.max.

Step 1: Use Math.round(4.6) to round to the nearest whole number.
Step 2: Use Math.max(45, 78, 62) to find the largest number.
Step 3: Print both results.

Task 3: Calculate the total cost from a price and a quantity stored as text.

Step 1: Store the price as '20.5' and the quantity as '3' (both are text).
Step 2: Convert the price with parseFloat and the quantity with parseInt.
Step 3: Multiply the two numbers and print the total with the label Total cost:.

6. Program / Queries / Commands:

Task 1

File: assignment5_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 5  Task: 1

console.log("parseInt:", parseInt('42px'));
console.log("parseFloat:", parseFloat('3.5'));
```

Task 2

File: assignment5_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 5  Task: 2

console.log(Math.round(4.6));
console.log(Math.max(45, 78, 62));
```

Task 3

File: assignment5_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 5  Task: 3

let priceText = '20.5';
let quantityText = '3';

let price = parseFloat(priceText);
let quantity = parseInt(quantityText);

let total = price * quantity;
console.log("Total cost:", total);
```

7. Input:

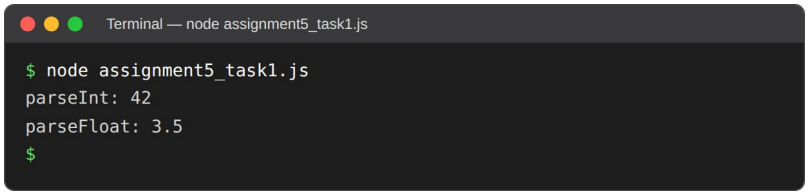
Task 1: Texts converted: '42px' and '3.5'

Task 2: Numbers used: 4.6 and 45, 78, 62

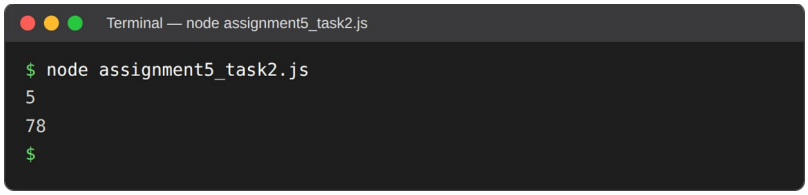
Task 3: price = '20.5', quantity = '3'

8. Output:

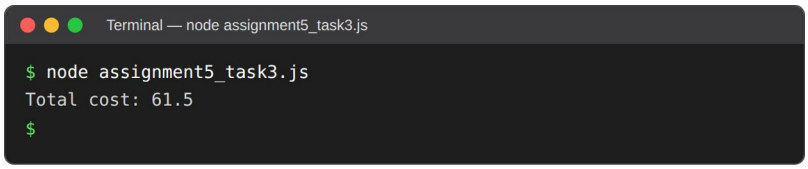
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. Text values were converted to numbers using parseInt and parseFloat, and Math.round and Math.max were used.

EXPERIMENT NO. 6

1. Experiment Title:

Conditions and the Ternary Operator

2. Aim:

To make decisions with if statements, an else if chain and the ternary operator.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

An if statement runs a block of code only when its condition is true, and else runs when the condition is false. An else if chain checks several conditions one after another and stops at the first one that is true. The ternary operator (condition ? valueIfTrue : valueIfFalse) is a short form of if-else that gives one of two values.

5. Algorithm / Procedure:

Task 1: Print Pass or Fail using if-else.

Step 1: Store the mark 35 in a variable.
Step 2: If mark >= 40 print 'Pass'.
Step 3: Otherwise print 'Fail'.

Task 2: Print Hot, Warm or Cold using an else if chain.

Step 1: Store the temperature 30 in a variable.
Step 2: If temperature >= 30 print 'Hot'.
Step 3: Else if temperature >= 20 print 'Warm'.
Step 4: Otherwise print 'Cold'.

Task 3: Print Eligible or Not Eligible using the ternary operator.

Step 1: Store the age 18 in a variable.
Step 2: Write the condition age >= 18 with the ternary operator.
Step 3: Print the result.

6. Program / Queries / Commands:

Task 1

File: assignment6_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 6  Task: 1

let mark = 35;

if (mark >= 40) {
  console.log('Pass');
} else {
  console.log('Fail');
}
```

Task 2

File: assignment6_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 6  Task: 2

let temperature = 30;

if (temperature >= 30) {
  console.log('Hot');
} else if (temperature >= 20) {
  console.log('Warm');
} else {
  console.log('Cold');
}
```

Task 3

File: assignment6_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 6  Task: 3

let age = 18;

let result = age >= 18 ? 'Eligible' : 'Not Eligible';
console.log(result);
```

7. Input:

Task 1: mark = 35

Task 2: temperature = 30

Task 3: age = 18

8. Output:

Task 1

Terminal — node assignment6_task1.js

```
$ node assignment6_task1.js
Fail
$
```

Task 2

Terminal — node assignment6_task2.js

```
$ node assignment6_task2.js
Hot
$
```

Task 3

Terminal — node assignment6_task3.js

```
$ node assignment6_task3.js
Eligible
$
```

9. Result:

The experiment was performed successfully. Decisions were made using if-else, an else if chain and the ternary operator.

EXPERIMENT NO. 7

1. Experiment Title:

Switch and If-Else Grades

2. Aim:

To choose one option from a few fixed choices with switch, and to print a grade with if-else.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

The switch statement compares one value with several case labels and runs the matching block. break stops the switch after a match, and default runs when no case matches. For ranges of values (such as marks) an if, else if, else chain is used because switch compares only exact values.

5. Algorithm / Procedure:

Task 1: Print the name of the day using switch.

Step 1: Store the day number 2 in a variable.
Step 2: Use switch with case 1, 2 and 3 to print Monday, Tuesday and Wednesday.
Step 3: Use default to print 'Invalid day'.

Task 2: Print a remark for a grade letter using switch.

Step 1: Store the grade letter 'B' in a variable.
Step 2: Use switch with case 'A', 'B' and 'C' for Excellent, Very good and Good.
Step 3: Use default to print 'Check the grade'.

Task 3: Print the grade for marks using if, else if and else.

Step 1: Store the marks 72 in a variable.
Step 2: If marks >= 80 print Grade: A.
Step 3: Else if marks >= 60 print Grade: B.
Step 4: Else if marks >= 40 print Grade: C.
Step 5: Otherwise print Grade: F.

6. Program / Queries / Commands:

Task 1

File: assignment7_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 7  Task: 1

let day = 2;

switch (day) {
  case 1:
    console.log('Monday');
    break;
  case 2:
    console.log('Tuesday');
    break;
  case 3:
    console.log('Wednesday');
    break;
  default:
    console.log('Invalid day');
}
```

Task 2

File: assignment7_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 7  Task: 2

let grade = 'B';

switch (grade) {
  case 'A':
    console.log('Excellent');
    break;
  case 'B':
    console.log('Very good');
    break;
  case 'C':
    console.log('Good');
    break;
  default:
    console.log('Check the grade');
}
```

Task 3

File: assignment7_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 7  Task: 3

let marks = 72;

if (marks >= 80) {
  console.log('Grade: A');
} else if (marks >= 60) {
  console.log('Grade: B');
} else if (marks >= 40) {
  console.log('Grade: C');
} else {
  console.log('Grade: F');
}
```

7. Input:

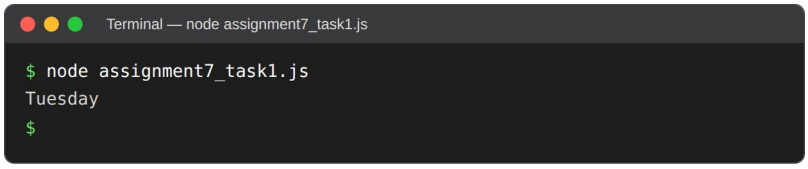
Task 1: day = 2

Task 2: grade = 'B'

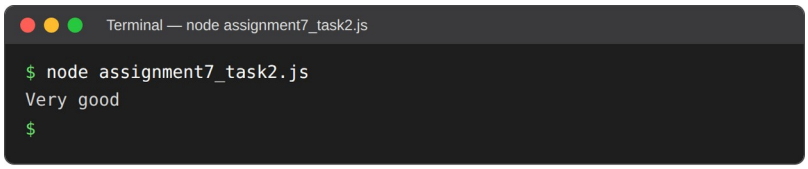
Task 3: marks = 72

8. Output:

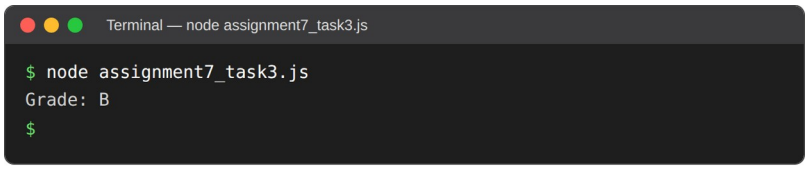
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. switch was used with case, break and default, and a grade was printed using an if, else if, else chain.

EXPERIMENT NO. 8

1. Experiment Title:

For, While and Do-While Loops

2. Aim:

To repeat a print with a for loop, a while loop and a do-while loop.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

A loop repeats a block of code. A for loop has the start value, the condition and the update in one line and is used when the number of repeats is known. A while loop checks the condition before each repeat. A do-while loop runs the block first and checks the condition after it, so it always runs at least once.

5. Algorithm / Procedure:

Task 1: Print the numbers 1 to 10 with a for loop.

Step 1: Start with i = 1.
Step 2: Repeat while i <= 10.
Step 3: Print i and increase it by 1 in each repeat.

Task 2: Count down from 3 to 1 with a while loop and print Liftoff!.

Step 1: Start with n = 3.
Step 2: While n >= 1 print n and decrease it by 1.
Step 3: After the loop print Liftoff!.

Task 3: Print 10, 7, 4, 1 with a do-while loop.

Step 1: Start with n = 10.
Step 2: Inside do: print n and subtract 3 from it.
Step 3: Repeat while n >= 1.

6. Program / Queries / Commands:

Task 1

File: assignment8_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 8  Task: 1

for (let i = 1; i <= 10; i++) {
  console.log(i);
}
```

Task 2

File: assignment8_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 8  Task: 2

let n = 3;

while (n >= 1) {
  console.log(n);
  n--;
}
console.log("Liftoff!");
```

Task 3

File: assignment8_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 8  Task: 3

let n = 10;

do {
  console.log(n);
  n -= 3;
} while (n >= 1);
```

7. Input:

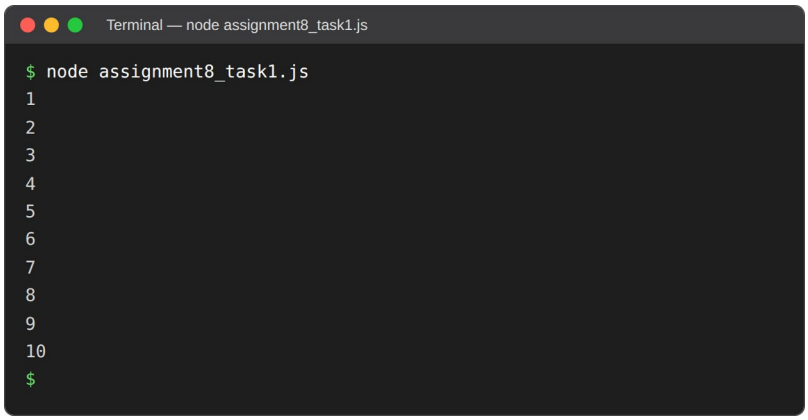
Task 1: Start value 1, end value 10.

Task 2: Start value n = 3.

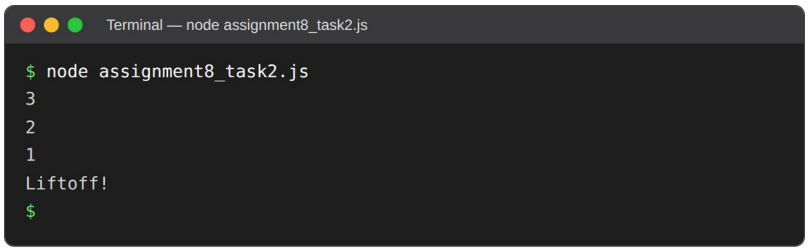
Task 3: Start value n = 10.

8. Output:

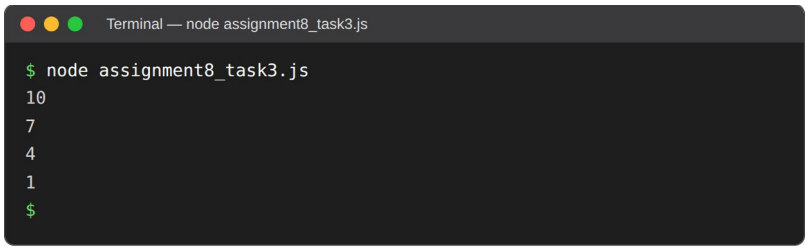
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. A for loop, a while loop and a do-while loop were used to repeat prints and the expected sequences were obtained.

EXPERIMENT NO. 9

1. Experiment Title:

Nested Loops and Continue

2. Aim:

To print a multiplication table and a star pattern with loops, and to skip a value with continue.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

A nested loop is a loop placed inside another loop; the inner loop runs fully for every repeat of the outer loop, which is useful for patterns made of rows and columns. The continue statement skips the rest of the current repeat and moves on to the next one.

5. Algorithm / Procedure:

Task 1: Print the multiplication table of 5 with a for loop.

Step 1: Run a for loop with i from 1 to 10.
Step 2: In each repeat print the line 5 x i = result.

Task 2: Print a triangle of stars with 4 rows using nested loops.

Step 1: The outer loop runs for each row, from 1 to 4.
Step 2: The inner loop adds one star to a text for each column up to the row number.
Step 3: Print the text of the row on one line.

Task 3: Print 1 to 10 but skip 5 with continue.

Step 1: Run a for loop with i from 1 to 10.
Step 2: If i is 5 use continue to skip the print.
Step 3: Otherwise print i.

6. Program / Queries / Commands:

Task 1

File: assignment9_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 9  Task: 1

for (let i = 1; i <= 10; i++) {
  console.log(`5 x ${i} = ${5 * i}`);
}
```

Task 2

File: assignment9_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 9  Task: 2

for (let row = 1; row <= 4; row++) {
  let line = "";
  for (let col = 1; col <= row; col++) {
    line += "*";
  }
  console.log(line);
}
```

Task 3

File: assignment9_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 9  Task: 3

for (let i = 1; i <= 10; i++) {
  if (i === 5) {
    continue;
  }
  console.log(i);
}
```

7. Input:

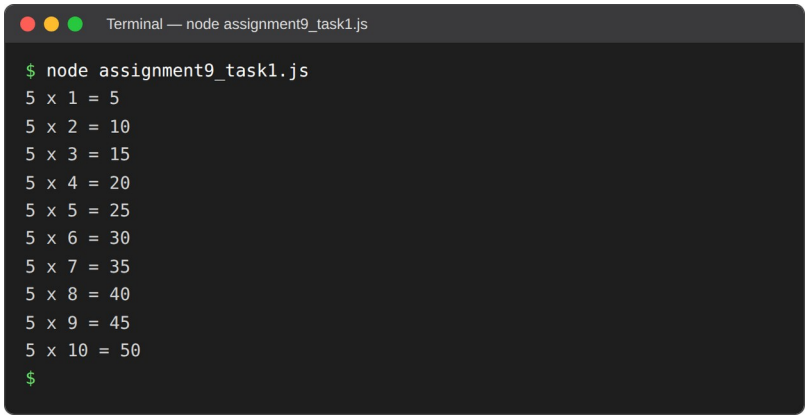
Task 1: Table number 5, from 1 to 10.

Task 2: Number of rows = 4.

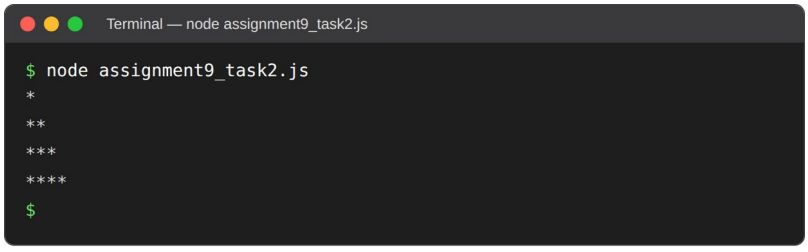
Task 3: Numbers from 1 to 10, skipped number = 5.

8. Output:

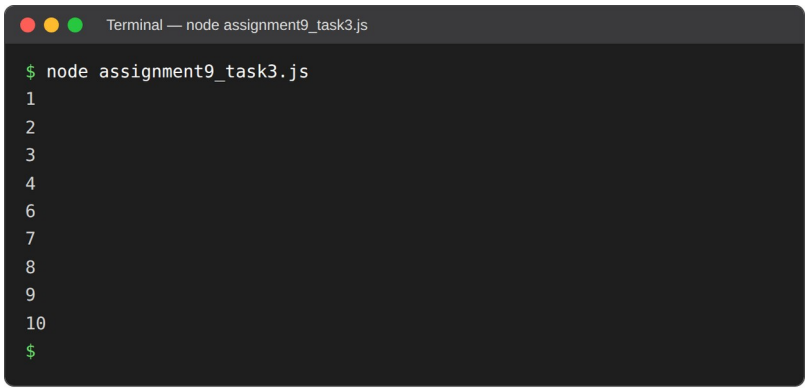
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. The multiplication table of 5, a star triangle using nested loops, and a number list that skips 5 with continue were printed.

EXPERIMENT NO. 10

1. Experiment Title:

Functions and Return Values

2. Aim:

To write functions with a parameter, a return value and a default value.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

A function is a named block of code that can be used again and again. Values given to a function are called parameters (or arguments when calling it). The return statement sends a value back to the place where the function was called. A default parameter (name = "Guest") is used when no value is passed.

5. Algorithm / Procedure:

Task 1: Write add(a, b) that returns the sum.

Step 1: Define function add(a, b).
Step 2: Return a + b from the function.
Step 3: Call add(4, 9) and print the result.

Task 2: Write isEven(n) that returns true or false.

Step 1: Define function isEven(n).
Step 2: Return true when n % 2 is 0, otherwise false.
Step 3: Print isEven(7) and isEven(10).

Task 3: Write greet(name) with the default value Guest.

Step 1: Define function greet(name = "Guest").
Step 2: Print "Hello, " + name + "!".
Step 3: Call greet() with no argument and then greet("Asha").

6. Program / Queries / Commands:

Task 1

File: assignment10_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 10  Task: 1

function add(a, b) {
  return a + b;
}

console.log(add(4, 9));
```

Task 2

File: assignment10_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 10  Task: 2

function isEven(n) {
  return n % 2 === 0;
}

console.log(isEven(7));
console.log(isEven(10));
```

Task 3

File: assignment10_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 10  Task: 3

function greet(name = "Guest") {
  console.log("Hello, " + name + "!");
}

greet();
greet("Asha");
```

7. Input:

Task 1: a = 4, b = 9

Task 2: n = 7 and n = 10

Task 3: No argument, then the name Asha.

8. Output:

Task 1

Terminal — node assignment10_task1.js

```
$ node assignment10_task1.js
13
$
```

Task 2

Terminal — node assignment10_task2.js

```
$ node assignment10_task2.js
false
true
$
```

Task 3

Terminal — node assignment10_task3.js

```
$ node assignment10_task3.js
Hello, Guest!
Hello, Asha!
$
```

9. Result:

The experiment was performed successfully. Functions with parameters, return values and a default parameter value were written and called.

EXPERIMENT NO. 11

1. Experiment Title:

Finding and Fixing Errors

2. Aim:

To use console.log and console.error, and to fix small mistakes in short programs.

3. Software / Tools Required:

JavaScript (ES6), HTML5, Visual Studio Code (editor), Google Chrome browser (DevTools Console)

4. Theory:

console.log() prints normal messages and console.error() prints error messages (shown in red in the console). Common mistakes are spelling a variable name wrongly (ReferenceError: the variable is not defined) and forgetting a closing bracket (SyntaxError). The error message and its line number in the console help to find and fix the mistake.

5. Algorithm / Procedure:

Task 1: Print messages with console.log and console.error.

Step 1: Print 'Starting program' with console.log.
Step 2: Print 'This is an error message' with console.error.
Step 3: Print 'Program finished' with console.log.

Task 2: Fix the program so that it prints Total: 225.

Step 1: Run the given program and read the error: ReferenceError: itemTree is not defined.
Step 2: Compare the name with the declarations and find that the variable is itemThree.
Step 3: Correct the spelling in the line that calculates total.
Step 4: Run the program again.

Task 3: Fix the program so that it prints 12.

Step 1: Run the given program and read the error: SyntaxError: Unexpected end of input.
Step 2: Check the brackets and find that the function double() is not closed.
Step 3: Add the closing } after the return statement.
Step 4: Run the program again.

6. Program / Queries / Commands:

Task 1

File: assignment11_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 11  Task: 1

console.log('Starting program');
console.error('This is an error message');
console.log('Program finished');
```

Task 2

Given program (with the mistake):

```
let itemOne = 120;
let itemTwo = 75;
let itemThree = 30;
let total = itemOne + itemTwo + itemTree;
console.log("Total:", total);
```

File: assignment11_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 11  Task: 2

let itemOne = 120;
let itemTwo = 75;
let itemThree = 30;
let total = itemOne + itemTwo + itemThree;   // fixed: itemTree -> itemThree
console.log("Total:", total);
```

Mistake found: the variable was written as itemTree but it was declared as itemThree. Changed itemTree to itemThree.

Task 3

Given program (with the mistake):

```
function double(n) {
  return n * 2;
console.log(double(6));
```

File: assignment11_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 11  Task: 3

function double(n) {
  return n * 2;
}   // fixed: closing brace added

console.log(double(6));
```

Mistake found: the closing brace } of the function double() was missing. Added it after the return statement.

7. Input:

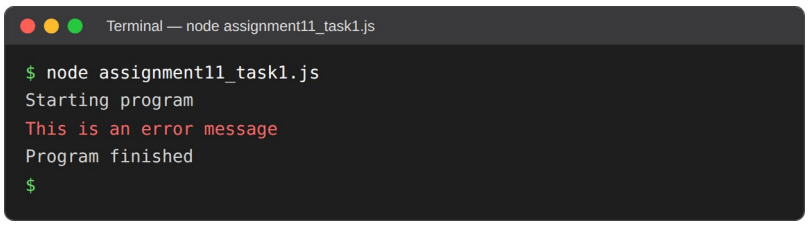
Task 1: No input is required.

Task 2: itemOne = 120, itemTwo = 75, itemThree = 30

Task 3: n = 6

8. Output:

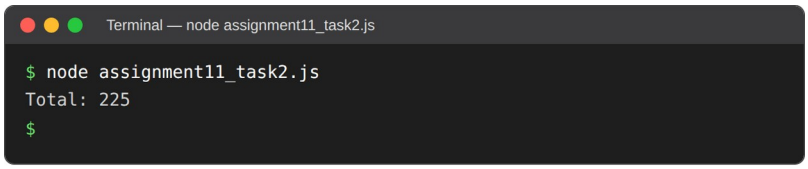
Task 1



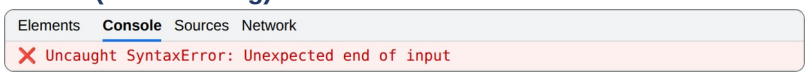
Task 2 (before fixing)



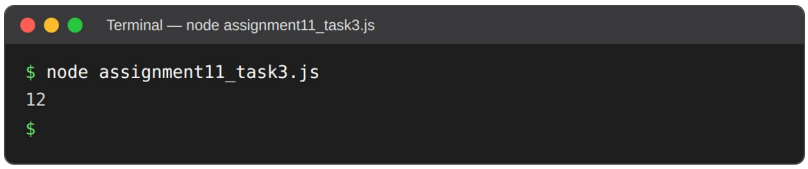
Task 2 (after fixing)



Task 3 (before fixing)



Task 3 (after fixing)



9. Result:

The experiment was performed successfully. console.log and console.error were used, the spelling mistake (itemTree) and the missing closing brace were found from the console error messages and fixed, and the correct output was obtained.

EXPERIMENT NO. 12

1. Experiment Title:

Calculator Functions

2. Aim:

To write calculator functions and to choose an operation with if-else and with switch.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

A calculator can be written as functions that take two numbers and return the result. A function calculate(op, a, b) receives the operator as text and decides which calculation to do. The choice can be made with an if / else if chain or with a switch statement; both give the same result, but switch is neater when comparing one value with many fixed choices.

5. Algorithm / Procedure:

Task 1: Write the functions multiply and divide.

Step 1: Define multiply(a, b) that returns a * b.
Step 2: Define divide(a, b) that returns a / b.
Step 3: Print multiply(12, 4) and divide(12, 4) with the labels.

Task 2: Write calculate(op, a, b) using if and else if.

Step 1: Define calculate(op, a, b).
Step 2: If op is '+' return a + b, else if '-' return a - b, else if '*' return a * b, else if '/' return a / b.
Step 3: Call the function for '+', '*' and '/' with 6 and 2 and print the results.

Task 3: Write calculate(op, a, b) using switch.

Step 1: Define calculate(op, a, b) and use switch (op).
Step 2: Write case '+', '-', '*' and '/' and return the result in each case.
Step 3: Call the function for '-' with 9 and 4, and for '*' with 3 and 3, and print the results.

6. Program / Queries / Commands:

Task 1

File: assignment12_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 12  Task: 1

function multiply(a, b) {
  return a * b;
}

function divide(a, b) {
  return a / b;
}

console.log("multiply:", multiply(12, 4));
console.log("divide:", divide(12, 4));
```

Task 2

File: assignment12_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 12  Task: 2

function calculate(op, a, b) {
  if (op === '+') {
    return a + b;
  } else if (op === '-') {
    return a - b;
  } else if (op === '*') {
    return a * b;
  } else if (op === '/') {
    return a / b;
  } else {
    return 'Invalid operator';
  }
}

console.log(calculate('+', 6, 2));
console.log(calculate('*', 6, 2));
console.log(calculate('/', 6, 2));
```

Task 3

File: assignment12_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 12  Task: 3

function calculate(op, a, b) {
  switch (op) {
    case '+':
      return a + b;
    case '-':
      return a - b;
    case '*':
      return a * b;
    case '/':
      return a / b;
    default:
      return 'Invalid operator';
  }
}

console.log(calculate('-', 9, 4));
console.log(calculate('*', 3, 3));
```

7. Input:

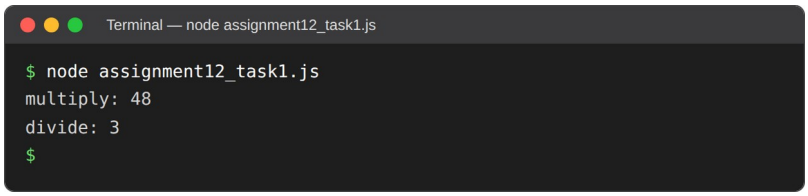
Task 1: a = 12, b = 4

Task 2: calculate('+', 6, 2), calculate('*', 6, 2), calculate('/', 6, 2)

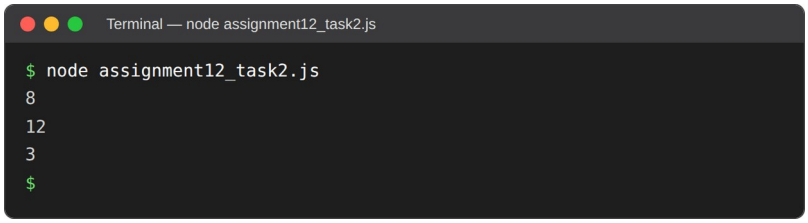
Task 3: calculate('-', 9, 4) and calculate('*', 3, 3)

8. Output:

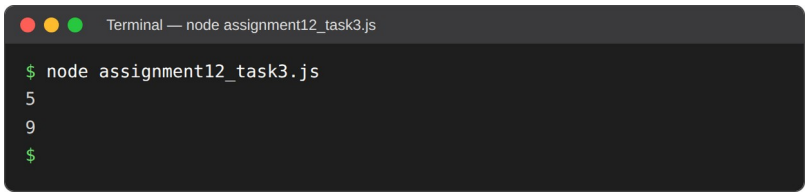
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. multiply and divide functions were written, and calculate(op, a, b) was made once with if-else and once with switch, giving the expected results.

EXPERIMENT NO. 13

1. Experiment Title:

The BMI Calculator Page Layout

2. Aim:

To build the HTML and CSS layout of a small web page that the next assignment uses.

3. Software / Tools Required:

HTML5, CSS3, Visual Studio Code (editor), Google Chrome browser

4. Theory:

HTML gives the structure of a page (title, heading, labels, inputs, button), and CSS gives the style. A CSS file is linked in the <head> with the <link> tag. Flexbox (display: flex) with justify-content and align-items places an element in the centre of the screen, and min-height: 100vh makes the body at least as tall as the screen. padding, border-radius and width style the card.

5. Algorithm / Procedure:

Task 1: Create a page with the title and heading BMI Calculator.

Step 1: Create index.html with the basic HTML structure.
Step 2: Write <title>BMI Calculator</title> in the head.
Step 3: Write <h1>BMI Calculator</h1> in the body.
Step 4: Open the file in the browser.

Task 2: Add the CSS link, the card, the two inputs, the Calculate button and the result paragraph.

Step 1: Create index.html with the title and the heading BMI Calculator.
Step 2: Link style.css in the head with the <link> tag.
Step 3: Add below the heading.
Step 4: Inside the card add the label and input (id weight), the label and input (id height), the Calculate button and an empty paragraph with id result.
Step 5: Create a simple style.css and open the page in the browser.

Task 3: Write the CSS that centres the card and styles it.

Step 1: Select body and set margin: 0, display: flex, justify-content: center, align-items: center, min-height: 100vh and background-color: #f2f2f2 (flex-direction: column keeps the heading above the card).
Step 2: Select .card and set background-color: white, padding: 24px, border-radius: 12px and width: 300px.
Step 3: Save as style.css (linked in the page from Task 2) and reload the page.

6. Program / Queries / Commands:

Task 1

File: index.html

```
<!-- Name: Deepak Yadav | Student ID: BCS2026491 | Assignment: 13  Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>BMI Calculator</title>
</head>
<body>
  <h1>BMI Calculator</h1>
</body>
</html>
```

Task 2

File: index.html

```
<!-- Name: Deepak Yadav | Student ID: BCS2026491 | Assignment: 13  Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>BMI Calculator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>BMI Calculator</h1>

  <div class="card">
    <label for="weight">Weight (kg)</label>
    <input type="number" id="weight">

    <label for="height">Height (m)</label>
    <input type="number" id="height">

    <button>Calculate</button>
    <p id="result"></p>
  </div>
</body>
</html>
```

File: style.css

```
/* Name: Deepak Yadav | Student ID: BCS2026491 | Assignment: 13  Task: 2 */
body {
  font-family: Arial, sans-serif;
  background: #f2f2f2;
  padding: 20px;
}

.card {
  background: white;
  padding: 24px;
  width: 300px;
  border-radius: 12px;
}

label {
  display: block;
  margin-top: 10px;
}

input {
  width: 100%;
  padding: 6px;
  box-sizing: border-box;
}

button {
  margin-top: 14px;
  padding: 8px 16px;
}
```

Task 3

File: style.css

```
/* Name: Deepak Yadav | Student ID: BCS2026491 | Assignment: 13  Task: 3 */
body {
  margin: 0;
  display: flex;
  flex-direction: column;     /* heading above the card */
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  background-color: #f2f2f2;
  font-family: Arial, sans-serif;
}

.card {
  background-color: white;
  padding: 24px;
  border-radius: 12px;
  width: 300px;
}

/* styling for the form controls inside the card */
label {
  display: block;
  margin-top: 10px;
}

input {
  width: 100%;
  padding: 6px;
  box-sizing: border-box;
}

button {
  margin-top: 14px;
  padding: 8px 16px;
}
```

7. Input:

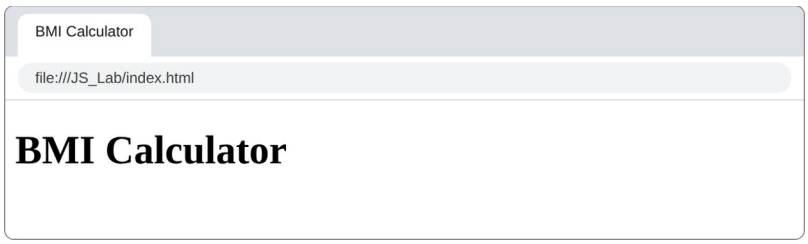
Task 1: No input is required.

Task 2: No input is required. The fields are left empty.

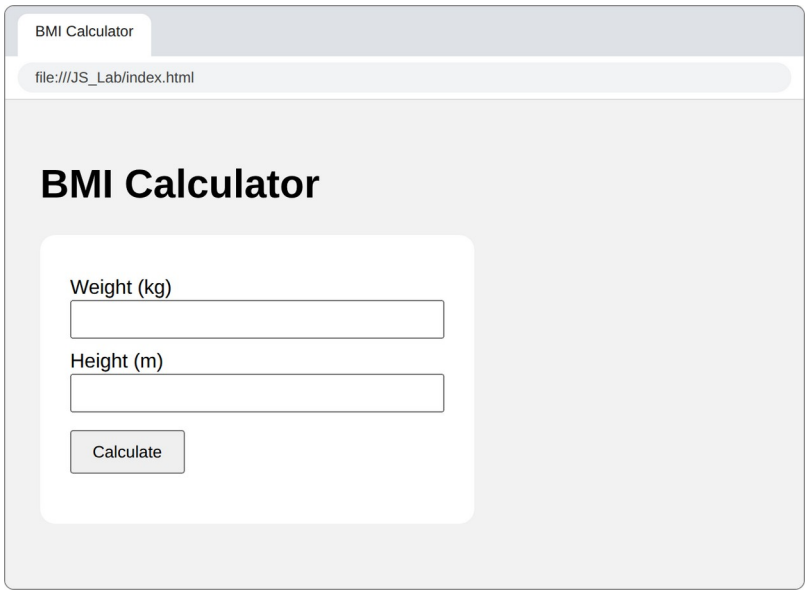
Task 3: No input is required.

8. Output:

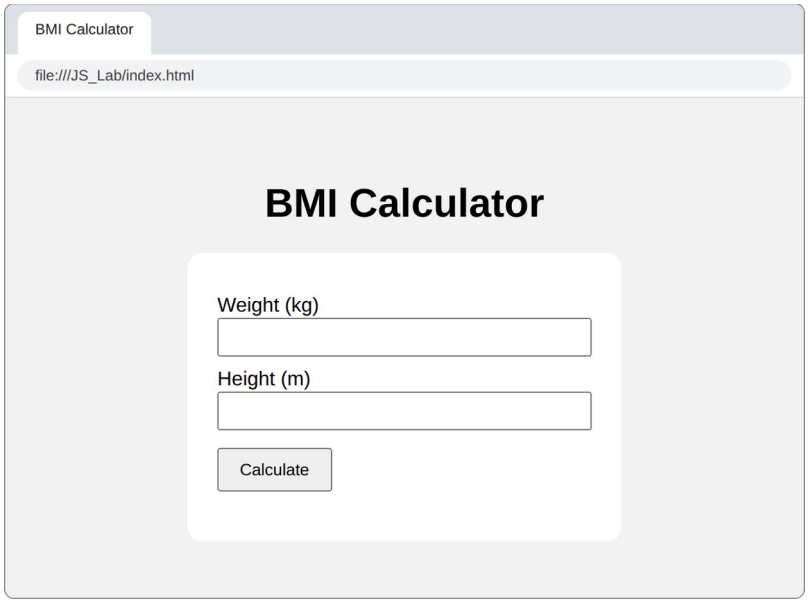
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. The BMI Calculator page was created with a heading, a card containing the weight and height inputs, a Calculate button and a result paragraph, and the card was centred and styled with CSS.

EXPERIMENT NO. 14

1. Experiment Title:

Script Linking and BMI Functions

2. Aim:

To link a script file to a page and to write functions that calculate a BMI and name its category.

3. Software / Tools Required:

JavaScript (ES6), HTML5, Visual Studio Code (editor), Google Chrome browser (DevTools Console)

4. Theory:

Body Mass Index (BMI) = weight (kg) / (height (m) x height (m)). A script file is linked to a page with <script src="script.js"></script> placed just before </body>. Math.round(bmi * 10) / 10 rounds a number to 1 decimal place. A second function uses comparisons to give the category: Underweight (below 18.5), Normal (below 25) or Overweight.

5. Algorithm / Procedure:

Task 1: Print Script loaded and link the script file to a page.

Step 1: Create script.js and write console.log("Script loaded").
Step 2: Create index.html and add <script src="script.js"></script> just before </body>.
Step 3: Open the page, press F12 and reload; check the Console tab.

Task 2: Write calculateBmi(weight, height).

Step 1: Define calculateBmi(weight, height).
Step 2: Calculate bmi = weight / (height * height).
Step 3: Return Math.round(bmi * 10) / 10 (1 decimal place).
Step 4: Print calculateBmi(70, 1.75).

Task 3: Write bmiCategory(bmi).

Step 1: Define bmiCategory(bmi).
Step 2: If bmi < 18.5 return 'Underweight'.
Step 3: Else if bmi < 25 return 'Normal'.
Step 4: Otherwise return 'Overweight'.
Step 5: Print bmiCategory(22.9).

6. Program / Queries / Commands:

Task 1

File: script.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 14  Task: 1

console.log("Script loaded");
```

File: index.html

```
<!-- Name: Deepak Yadav | Student ID: BCS2026491 | Assignment: 14  Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>BMI Calculator</title>
</head>
<body>
  <h1>BMI Calculator</h1>

  <script src="script.js"></script>
</body>
</html>
```

Task 2

File: assignment14_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 14  Task: 2

function calculateBmi(weight, height) {
  let bmi = weight / (height * height);
  return Math.round(bmi * 10) / 10;
}

console.log(calculateBmi(70, 1.75));
```

Task 3

File: assignment14_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 14  Task: 3

function bmiCategory(bmi) {
  if (bmi < 18.5) {
    return 'Underweight';
  } else if (bmi < 25) {
    return 'Normal';
  } else {
    return 'Overweight';
  }
}

console.log(bmiCategory(22.9));
```

7. Input:

Task 1: No input is required.

Task 2: weight = 70 kg, height = 1.75 m

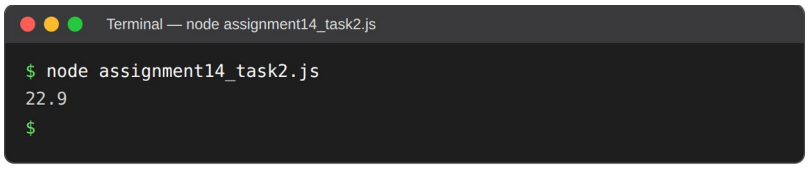
Task 3: bmi = 22.9

8. Output:

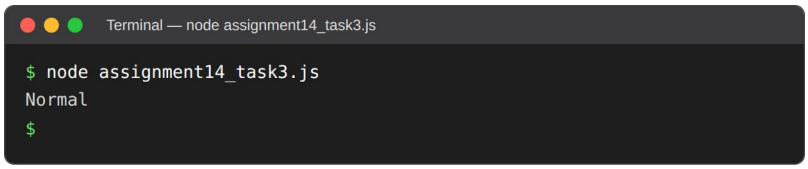
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. script.js was linked to the page (Script loaded appeared in the console), calculateBmi(70, 1.75) gave 22.9 and bmiCategory(22.9) gave Normal.

EXPERIMENT NO. 15

1. Experiment Title:

Testing Normal, Boundary and Invalid Inputs

2. Aim:

To test small functions with normal, boundary and invalid inputs and to read the results.

3. Software / Tools Required:

JavaScript (ES6), Node.js, Visual Studio Code (editor)

4. Theory:

Testing means running a function with different inputs to check that it works. A normal input is a typical value, a boundary input is a value exactly at the limit of a condition (such as 18 for age >= 18), and an invalid input is a value outside the allowed range (such as a mark of 101). Boundary values are tested because mistakes such as using > instead of >= show up only there.

5. Algorithm / Procedure:

Task 1: Write isAdult(age) and test 17, 18 and 30.

Step 1: Define isAdult(age) that returns age >= 18.

Step 2: Test the boundary values 17 and 18, and the normal value 30.

Task 2: Write isPositive(n) and test 0, 1 and -5.

Step 1: Define isPositive(n) that returns n > 0.

Step 2: Test the boundary value 0, the value 1 and the negative value -5.

Task 3: Write checkMark(mark) and test -1, 0 and 101.

Step 1: Define checkMark(mark).

Step 2: If mark < 0 or mark > 100 return 'Invalid', otherwise return 'Valid'.

Step 3: Test -1 and 101 (invalid) and 0 (boundary).

6. Program / Queries / Commands:

Task 1

File: assignment15_task1.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 15  Task: 1

function isAdult(age) {
  return age >= 18;
}

console.log(isAdult(17));
console.log(isAdult(18));
console.log(isAdult(30));
```

Task 2

File: assignment15_task2.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 15  Task: 2

function isPositive(n) {
  return n > 0;
}

console.log(isPositive(0));
console.log(isPositive(1));
console.log(isPositive(-5));
```

Task 3

File: assignment15_task3.js

```
// Name: Deepak Yadav
// Student ID: BCS2026491
// Assignment: 15  Task: 3

function checkMark(mark) {
  if (mark < 0 || mark > 100) {
    return 'Invalid';
  }
  return 'Valid';
}

console.log(checkMark(-1));
console.log(checkMark(0));
console.log(checkMark(101));
```

7. Input:

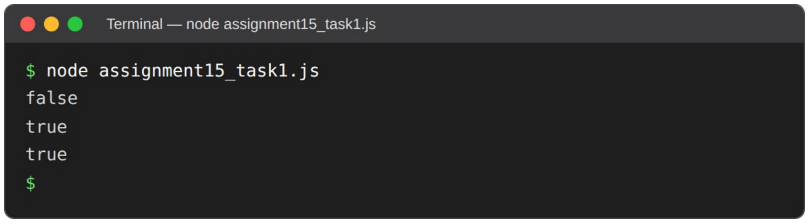
Task 1: age = 17 (below limit), 18 (boundary), 30 (normal)

Task 2: n = 0 (boundary), 1 (normal), -5 (negative)

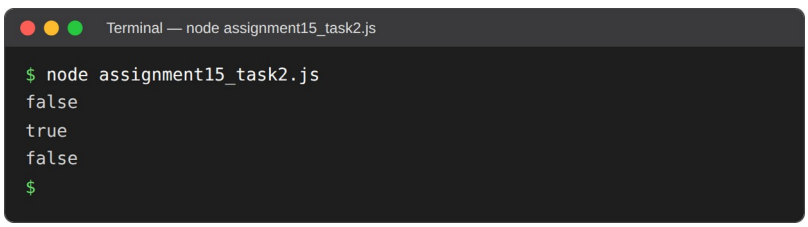
Task 3: mark = -1 (invalid), 0 (boundary), 101 (invalid)

8. Output:

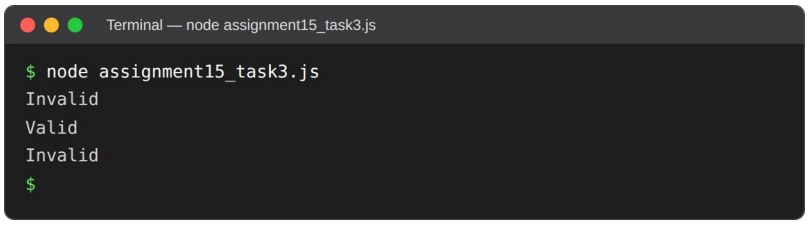
Task 1



Task 2



Task 3



9. Result:

The experiment was performed successfully. isAdult, isPositive and checkMark were tested with normal, boundary and invalid inputs and all outputs matched the expected results.